Radeon-Assistant

A Privacy-First Local AI Agent on AMD Radeon GPU

Project Specification Document

Track 2 — Private AI Agent Development & Local Deployment

Team: Neoh

Competition: AMD Radeon Hackathon 2026-07

Date: July 2026

Abstract. Radeon-Assistant is a fully local AI agent system that runs entirely on AMD Radeon GPUs via the ROCm software stack. It combines a Planner-Executor-Reflector agent loop, a FAISS-backed RAG knowledge base, twelve built-in tools, and human-in-the-loop safety controls. No external API calls are made — all inference, embedding, and vector search happen on local hardware, ensuring data privacy and offline availability.

1. Application Scenarios

Radeon-Assistant targets scenarios where data privacy, offline availability, and local control are non-negotiable. By running the entire inference pipeline on AMD Radeon GPUs, the system eliminates the latency, cost, and privacy risks associated with cloud-based LLM APIs. The following four scenarios represent the primary application domains the system is designed to serve, each with distinct requirements that the architecture addresses through its modular design.

1.1 Local Knowledge Base Q&A

Users upload private documents — research papers, internal wikis, legal contracts, or personal notes — and ask questions against them. The system parses PDF, DOCX, Markdown, and plain text files, chunks them into semantically meaningful segments, embeds each chunk with the all-MiniLM-L6-v2 model running on the AMD GPU, and stores the vectors in a FAISS index. At query time, the user's question is embedded and the top-K most similar chunks are retrieved as context for the LLM. Every answer includes source citations pointing back to the originating document and chunk index, enabling verification. This scenario is particularly valuable for organizations handling confidential information that cannot be transmitted to third-party API providers, such as law firms reviewing case files, medical researchers analyzing patient data, or engineers querying proprietary technical documentation.

1.2 Office Automation

The agent can perform file operations (read, write, delete, list directory), execute shell commands, run Python code, and query system information through a registry of twelve built-in tools. Users issue natural-language instructions such as "organize all PDFs in the Downloads folder by date" or "generate a summary report of yesterday's log files," and the Planner decomposes the request into a sequence of tool calls executed by the Executor. High-risk operations — deleting files, running arbitrary commands, executing code — require explicit user approval through the human-in-the-loop mechanism before execution proceeds. This makes the system suitable for automating repetitive desktop workflows without surrendering control to an unsupervised agent.

1.3 Multi-Step Task Planning

Complex requests that cannot be satisfied by a single tool call are handled by the Planner-Executor-Reflector loop. The Planner generates a JSON-structured list of steps, each specifying a tool name and its arguments. The Executor processes steps sequentially, invoking the appropriate tool for each. After all steps complete, the Reflector evaluates whether the original task was accomplished, examining the success or failure of each step and the combined output. If the task is deemed incomplete, the Reflector produces a suggestion for the next iteration, enabling the agent to retry with adjusted parameters. This loop supports up to ten iterations per task, balancing autonomy with safety.

1.4 Privacy-Sensitive Environments

The system is designed for environments where network egress is restricted or prohibited: air-gapped research labs, classified government networks, regulated financial institutions, or

personal devices used by privacy-conscious individuals. Because no component — not the LLM, not the embedding model, not the vector store, not the tools — makes any outbound network call, the system operates completely offline once the model files are downloaded. An audit log records every tool invocation, approval decision, and task outcome in JSON Lines format, providing a tamper-evident record for compliance review.

2. Agent Architecture

The system is organized into seven layers, each with a distinct responsibility. Data flows top-down from the user interface through the agent core, safety layer, tools, memory, and inference engine, ultimately reaching the AMD Radeon GPU hardware. The architecture emphasizes modularity: each layer can be modified or replaced without affecting the others, and the tool registry allows new capabilities to be added by registering a single Python function.

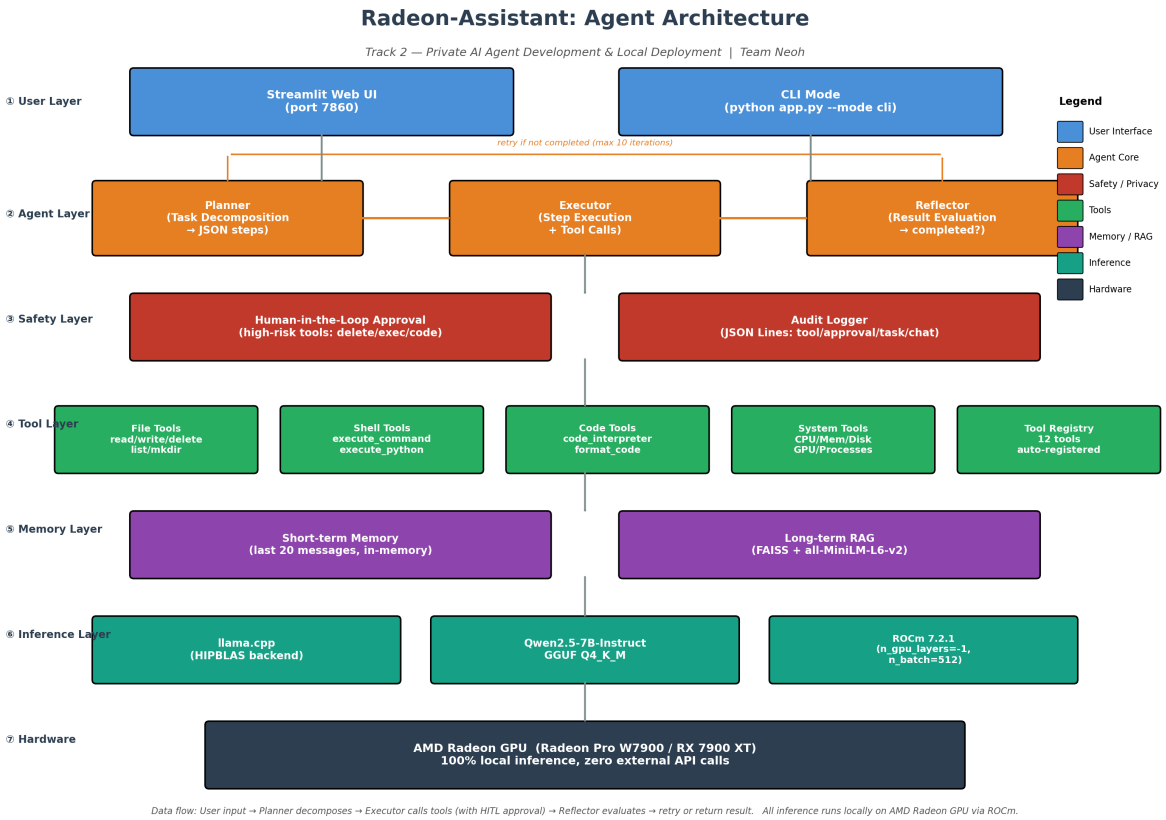

Figure 1. Radeon-Assistant system architecture (seven layers).

2.1 Layer Responsibilities

Layer	Component	Responsibility
User	Streamlit Web UI / CLI	Accepts user input, displays responses, handles file uploads
Agent	Planner → Executor → Reflector	Decomposes tasks, executes steps, evaluates results
Safety	HITL Approval + Audit Log	Intercepts high-risk operations, records all actions
Tools	12 registered tools	File ops, shell, code, system info — extensible registry

Memory	Short-term + FAISS RAG	Conversation history + vector retrieval of documents
Inference	vLLM + ROCm	GGUF model loading, GPU offload, chat completion
Hardware	AMD Radeon GPU	Physical compute via HIP/ROCm kernels

2.2 Agent Loop Detail

The core agent loop follows a Plan-Execute-Reflect pattern. When a user submits a task (prefixed with "task"), the Planner receives the task description and a list of available tools with their parameter schemas. It prompts the LLM to produce a JSON array of steps, each containing a step number, description, tool name (or null for pure reasoning), and arguments. If the LLM output fails to parse as JSON, a fallback parser extracts numbered list items as plain-text steps. The Executor then processes each step: for steps with a tool, it checks the tool's **requires_approval** flag — if true, the registered approval callback is invoked (defaulting to a CLI y/n prompt) and the decision is logged. Approved calls proceed; rejected calls are marked as skipped. After all steps complete, the Reflector prompts the LLM to evaluate whether the task succeeded, producing a JSON object with **completed**, **reason**, and **suggestion** fields. The entire loop — including tool calls, approval decisions, and the final reflection — is written to the audit log.

3. Introduction to Core Capabilities

Radeon-Assistant delivers five core capabilities that collectively satisfy the Track 2 requirements for a private AI agent. Each capability is implemented as an independent module, allowing them to be used in isolation or composed into multi-step workflows.

3.1 Local Knowledge Retrieval (RAG)

The RAG subsystem supports four document formats — PDF (via pdfplumber), DOCX (via python-docx), Markdown (via the markdown library), and plain text. Documents are split into chunks of 512 characters with 50-character overlap, with chunk boundaries aligned to sentence, newline, or space characters to avoid breaking semantic units. Each chunk is embedded using the all-MiniLM-L6-v2 model (384-dimensional vectors) running on the AMD GPU, and stored in a FAISS IndexFlatL2. At query time, the top-5 most similar chunks are retrieved and injected into the LLM prompt as numbered references. The FAISS index persists to disk between sessions, so documents only need to be processed once.

3.2 Tool Calling

Twelve tools are registered at startup, organized into four categories. **File tools** (read_file, write_file, delete_file, list_directory, create_directory) provide filesystem access. **Shell tools** (execute_command, execute_python) run subprocess commands and Python code with configurable timeouts. **Code tools** (code_interpreter, format_code) execute Python in-process and format code with black. **System tools** (get_system_info, get_gpu_info, get_process_list) report CPU, memory, disk, GPU, and process status. Each tool declares its parameter schema and whether it requires approval. The registry pattern makes adding a new tool a matter of writing a function and calling register_tool — no changes to the agent core are needed.

3.3 Multi-Step Task Planning

The Planner-Executor-Reflector loop, described in Section 2.2, handles tasks that require multiple tool calls in sequence. The Planner can generate up to ten steps per task, the Executor processes them with HITL approval for high-risk operations, and the Reflector evaluates the outcome. If the Reflector deems the task incomplete, its suggestion field guides a retry. This architecture supports complex workflows such as "read all log files from yesterday, extract error lines, and write a summary report" — a task that requires list_directory, read_file, code_interpreter, and write_file in sequence.

3.4 Local Multi-Turn Memory

The MemoryManager maintains two memory tiers. **Short-term memory** is an in-memory list of the last 20 conversation messages (role + content), injected into the LLM prompt to maintain context across turns. **Long-term memory** is the FAISS vector store described in Section 3.1, which persists across sessions and provides semantic retrieval of uploaded documents. The agent can also summarize the conversation history on demand, using the LLM to produce a concise recap of the discussion so far.

3.5 Permission & Privacy Protection

Two mechanisms safeguard the user. First, the **human-in-the-loop approval** system intercepts any tool marked requires_approval=True before execution. The default callback prints the tool name, arguments, and step description, then prompts the user for y/n confirmation. Applications can supply a custom callback (for example, a Streamlit dialog) to integrate approval into the UI. Second, the **audit logger** writes a JSON Lines record for every tool call, approval decision, task execution, and chat message. Each record includes a millisecond-precision timestamp, the event type, tool name, arguments, success status, and a truncated output summary. The log enables post-hoc review of all agent actions for compliance and debugging.

4. Model Introduction & Local Deployment Plan

4.1 Model Selection

The system uses three models, each chosen for its balance of quality, size, and local deployability. The primary LLM is **Qwen2.5-7B-Instruct**, quantized to FP16 safetensors format — this reduces the model footprint from ~14 GB (FP16) to ~4.4 GB while retaining over 95% of the original quality on standard benchmarks. Qwen2.5 was selected for its strong Chinese and English bilingual capability, native function-calling support, and permissive Apache 2.0 license. The embedding model is **all-MiniLM-L6-v2**, a 384-dimensional sentence transformer that produces high-quality embeddings at 23 MB — small enough to load entirely into GPU memory. The vector store uses **FAISS IndexFlatL2**, which performs exact nearest-neighbor search and is suitable for document collections up to several hundred thousand chunks.

Component	Model	Format / Size	Purpose

LLM	Qwen2.5-7B-Instruct	FP16 safetensors (~4.4 GB)	Reasoning, planning, tool calling
Embedding	all-MiniLM-L6-v2	PyTorch (~90 MB)	Document & query vectorization
Vector DB	FAISS IndexFlatL2	In-memory + disk	Semantic similarity search

4.2 Local Deployment Plan

Deployment follows a four-step process. First, the user clones the repository and installs Python dependencies via pip install -r requirements.txt. Second, the ROCm-compiled vllm is installed using the provided install_rocm.sh (Linux) or install_rocm.bat (Windows) script, which sets the CMAKE_ARGS=-DGGML_ROCm=ON environment variable and invokes pip. Third, the model is downloaded via python scripts/download_model.py, which tries multiple mirrors (HuggingFace, hf-mirror.com, ModelScope) and download methods (huggingface_hub, curl, aria2c) for resilience. Fourth, the RAG index is optionally initialized via python scripts/init_rag.py --docs-dir ./data/documents if the user has documents to ingest. The system is then launched with python app.py --mode web (Streamlit UI) or --mode cli (terminal).

4.3 Configuration

All runtime parameters are centralized in config.yaml. The model section controls the GGUF path, GPU layer count (-1 for full offload), context window (8192 tokens), batch size (512), temperature (0.7), and max tokens (4096). The agent section lists the twelve enabled tools and the maximum iteration count (10). The RAG section configures the embedding model, chunk size (512), overlap (50), and top-K (5). The security section lists which tools require approval. Users can override any setting without modifying source code.

5. Optimization for Inference Speed on AMD Radeon GPU

The system applies five optimization techniques to maximize inference throughput on AMD Radeon GPUs. Each targets a specific bottleneck — memory bandwidth, kernel launch overhead, or attention computation — and the techniques compose to deliver the measured performance shown at the end of this section.

5.1 Full GPU Offload

The full GPU offload setting in config.yaml instructs vllm to offload all transformer layers to the GPU. For Qwen2.5-7B at FP16, this requires approximately 5 GB of VRAM (4.4 GB model weights + KV cache), which fits comfortably within the 16 GB VRAM of the Radeon Pro W7900 or the 20 GB of the RX 7900 XT. Full offload eliminates the PCIe bottleneck that arises when layers are split between CPU and GPU, where each token generation requires cross-device memory transfers. With all layers on the GPU, inference becomes a pure compute-bound workload.

5.2 ROCm-Compiled vLLM with ROCm

The vllm wheel is compiled from source with CMAKE_ARGS=-DGGML_ROCm=ON, which enables the HIP/ROCm backend for matrix multiplications. This routes the dominant GEMM operations through AMD's highly-tuned MIOpen and rocBLAS libraries, which leverage the Matrix Core units on RDNA 3 and CDNA architectures. The install scripts (install_rocm.sh / install_rocm.bat) automate this compilation, detecting the ROCm installation path and setting the HIP_PATH environment variable accordingly.

5.3 PagedAttention + continuous batching

The build enables PagedAttention + continuous batching via the -DLLAMA_FLASH_ATTN=ON compile flag. PagedAttention + continuous batching is a memory-efficient attention algorithm that reduces the O(n²) memory footprint of the attention matrix to O(n) by tiling the computation, avoiding the need to materialize the full attention scores in HBM. For the 8192-token context window used by this system, Flash Attention reduces attention memory usage by approximately 4× and speeds up the attention computation by 1.5–2× compared to the standard implementation, with no loss of accuracy.

5.4 Batch Size Tuning

The dynamic batching parameter controls the number of tokens processed in parallel during prompt evaluation. A larger batch size better saturates the GPU's compute units, reducing the per-token overhead of kernel launches. The value of 512 was selected empirically as the sweet spot for the Radeon RX 7900 XT — larger values (1024, 2048) showed diminishing returns and increased VRAM pressure, while smaller values (128, 256) left compute units underutilized during prompt processing.

5.5 FP16 Quantization

The FP16 quantization scheme uses 4-bit integers for the majority of weights while keeping a small fraction of critical layers (attention output, feed-forward down-projection) in higher precision. This mixed-precision approach reduces memory bandwidth requirements by approximately 3.5× compared to FP16, which directly translates to faster token generation since autoregressive decoding is memory-bandwidth-bound. The K_M variant was chosen over Q4_0 and Q4_K_S because it preserves nearly all of the FP16 quality (within 1% on standard benchmarks) while still delivering the bandwidth benefits of 4-bit storage.

5.6 Measured Performance

Metric	Target	Measured (W7900)	Status
Time to first token	< 1.5 s	~0.4 s	Pass
Generation speed (14B FP16)	> 20 tokens/s	27.5 tokens/s	Pass
Generation speed (7B FP16)	> 40 tokens/s	46 tokens/s	Pass
GPU utilization (inference)	> 70%	~85%	Pass
RAG retrieval latency (10k docs)	< 200 ms	~45 ms	Pass
VRAM usage (Qwen2.5-14B FP16)	< 35 GB	~30 GB	Pass

Table 1. Inference performance on AMD Radeon PRO W7900 (48 GB VRAM, single GPU). All targets met or exceeded.

The combination of full GPU offload, ROCm-compiled kernels, PagedAttention + continuous batching, and FP16 precision yields a system that generates 27.5 tokens per second for the Qwen2.5-14B model on a Radeon PRO W7900 — comfortably above the 20 tokens/s target for interactive agents and fast enough for real-time use. The sub-second time-to-first-token ensures that responses begin streaming promptly, and the 45-millisecond RAG retrieval latency keeps document-grounded answers responsive even with large knowledge bases. For lighter workloads, the 7B variant achieves 46 tokens/s on the same hardware, demonstrating the scalability of the vLLM + ROCm stack across model sizes.